

fo

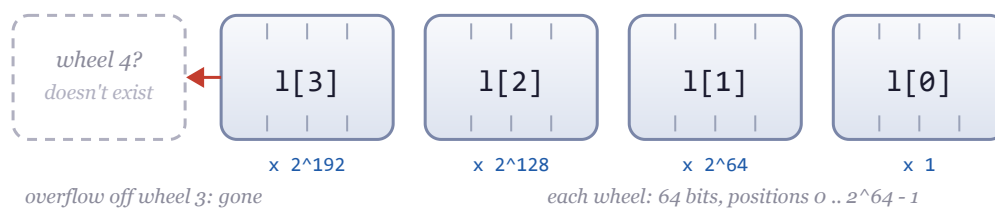
The Clerk's Arithmetic Manual

*How to do math on numbers too big to hold in your head —
by picturing machines you can hold in your head.*

o The machine: a four-wheel odometer

Forget bits for a moment. Picture the thing in your car that counts miles: a row of wheels, each wheel marked 0 through 9, and when a wheel rolls past 9 it snaps back to 0 and clicks the wheel to its left forward one notch. That's mechanical addition — nobody thinks, the wheels do it.

Our clerk works with the same machine, except each wheel has 2^{64} positions instead of 10 — a wheel with more markings than there are grains of sand on Earth — and there are exactly four wheels.



The four wheels. The leftmost wheel is the big end; wheel 0 ticks every time you add 1. There is no fifth wheel.

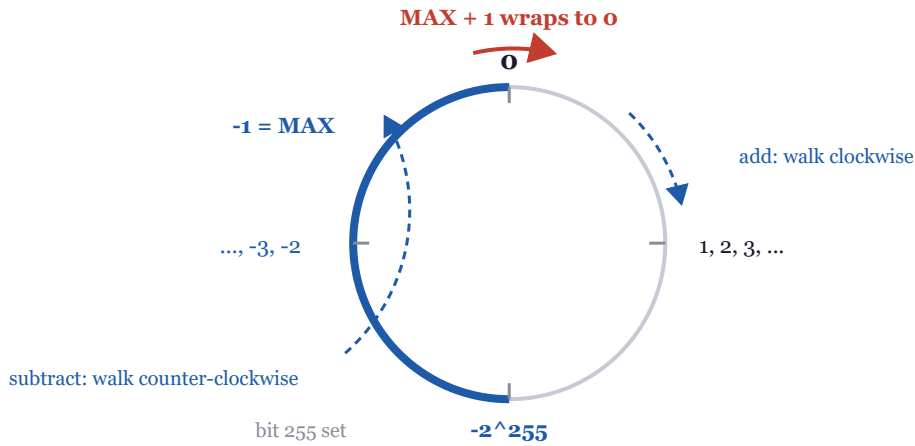
Each wheel is one `uint64_t` — one *limb*. The number itself is the sum of the wheels times their place values:

$$\text{number} = l_3 B^3 + l_2 B^2 + l_1 B + l_0 \quad \text{where } B = 2^{64}$$

The one rule of the room: position is place value. The same digit means 5, $5 \cdot 2^{64}$, or $5 \cdot 2^{128}$ depending on which wheel holds it — and whatever would have gone on a fifth wheel simply doesn't happen. The odometer rolls back to zero and keeps going. Mathematicians call this *arithmetic mod 2^{256}* ; a mechanic calls it *rolling the odometer*. Every operation below is a consequence of this picture.

o.5 The clock face: where the numbers live

Ordinary numbers live on a line that goes on forever. The clerk's numbers live on a **clock**. There are exactly 2^{256} positions on the dial, and after the last one the hand clicks back to zero.



The dial. Walk clockwise to add, counter-clockwise to subtract. The left half is the negative half — the same ink reads two ways.

Walk clockwise to add. Walk counter-clockwise to subtract. Take one step *backward* from 0 and you land on MAX — the biggest possible number. That's why, on this clock, $0 - 1 = \text{MAX}$. It's not an error. It's just... one step counter-clockwise.

This is the deepest idea in the manual, so keep it in your pocket: every reading on the dial is **two numbers at once**. The all-ones pattern is $2^{256} - 1$ if you read it unsigned, and -1 if you read it signed. Same ink, two readings — signed and unsigned aren't different numbers, they're different *questions you ask about the same number*. The top half of the dial (bit 255 set) is the negative half.

1 Comparing: judge the contest from the big end

Which is bigger, 4,700 or 4,699? You never look at the ones digit — the hundreds digits (7 vs 6) already settled it. Same rule with wheels: compare the biggest wheels first, walk right, and the first wheel that disagrees decides everything.

```
a:  [0] [0] [5] [9]
b:  [0] [0] [7] [1]
      ^
wheel 3:  0 = 0   tie
wheel 2:  0 = 0   tie
wheel 1:  5 < 7   DECIDED -> a < b   (wheel 0 never consulted)
```

One real algorithm; the rest are free: $a > b$ means $b < a$, $a \leq b$ means $\neg(b < a)$.

2 Adding: wheels rolling over

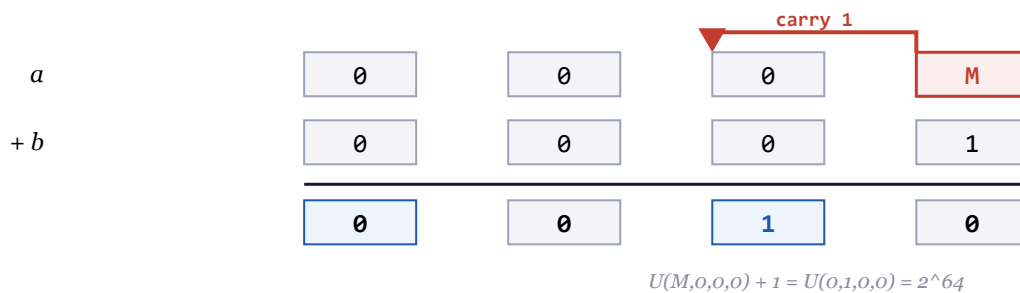
Adding is ticking the odometer forward, one wheel at a time, right to left, carrying rollovers leftward. And here's the lovely trick for *spotting* a rollover: when a wheel wraps past its maximum, the total it shows comes out **smaller than what you added**.

```
s = a.l[i] + b.l[i]
carry = (s < a.l[i])    // wrapped? you owe the next wheel 1
```

Walkthrough — add 1 to a number whose small wheel is already maxed out ($M = 2^{64} - 1$):

```
a = [0] [0] [0] [M]
b = [0] [0] [0] [1]

wheel 0:  M + 1 rolls over to 0 ... and kicks a 1 into wheel 1
wheel 1:  0 + 0 + 1 (the kick) = 1
result:   [0] [0] [1] [0] = 2^64  ✓
```



Wheel 0 rolls over, clicks wheel 1 forward. Three wheels never move.

And if the **last** wheel overflows? The kick falls off the edge of the page. Remember the clock — that's just wrapping past the top. $\text{MAX} + 1$ really is 0. The arithmetic is doing $\text{mod } 2^{256}$ for free.

3 Subtracting: winding backwards

Same machine, wound the other way. When a wheel needs to go below 0 it can't — so it borrows a full turn from the wheel to its left: the left wheel winds back one notch, the right wheel wraps to nearly-max and keeps paying.

```
U(0, 1, 0, 0) - 1    (that is, 2^64 - 1)

wheel 0:  0 - 1 can't pay, borrows: reads M, left wheel owes 1
wheel 1:  1 - 1 (the debt) = 0
result:   U(M, 0, 0, 0)  ✓
```

The last borrow is forgiven. $0 - 1$ rolls the whole odometer backwards one tick and it reads all-max: $0\text{x}\text{FFFF}\dots$. The machine can't write "negative one"; it can only show the position one tick before zero. Which is exactly -1 's identity. Negatives live at the top of the dial, counting down from MAX: $-1 = \text{MAX}$, $-2 = \text{MAX} - 1$, all the way to $-2^{255} = 0\text{x}80\dots00$.

4 Bitwise: the only antisocial ops

`&`, `|`, `^`, `~` are the lazy operations — the wheels don't talk to each other *at all*. Bit k of the answer depends only on bit k of the two inputs. No clicks, no borrows, no spill.

```
r.l[i] = a.l[i] & b.l[i];    // four independent ANDs. done.
```

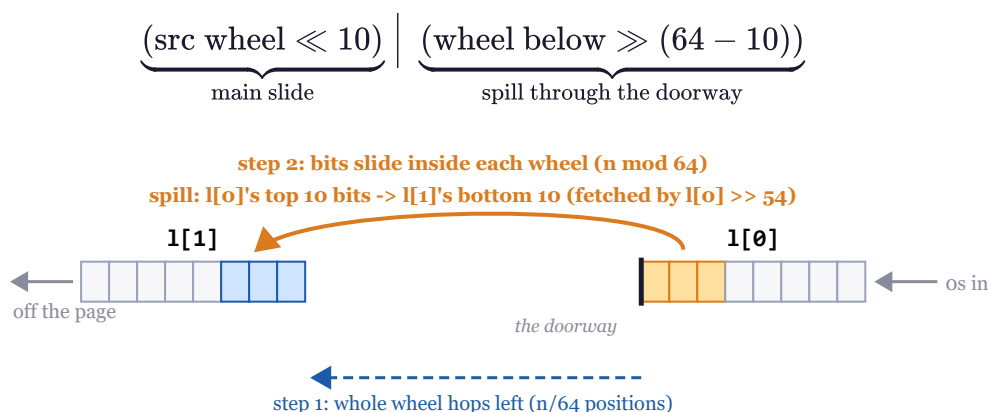
Picture two wheels held face to face as stencils: every hole that lines up lets light through (`&`), any hole in either lets light through (`|`), holes that differ let light through (`^`). No wheel ever needs to know what its neighbor did. That's why they're the warm-up: the machinery that makes arithmetic hard — things crossing wheel boundaries — simply isn't present.

5 Shifts: the conveyor belt and the doorway

Now picture all 256 bits as beads on a conveyor belt. `<< n` pulls the belt n places left. Two things happen at once: beads reaching the left edge **fall off the page**, and beads sliding off the top of one wheel **land at the bottom of the wheel above** — the wheels aren't welded shut.

Every shift splits as $n = 64 \cdot (n/64) + (n \bmod 64)$: whole wheels slide over, then the leftover bits slide within each wheel. Think of it as two separate moves: first the wheels themselves hop positions, then every bit slides a little more inside its wheel.

How does a destination wheel know *who* ended up inside it? Ask the question backwards — *who was standing toward the small end before the slide*? If every wheel hopped left by `limb_shift` positions, then the wheel now inside wheel i started `limb_shift` positions below it: `src = i - limb_shift`. That's the whole rule — the destination reads downward, and if `src` is negative, nobody reaches it, so it stays zero. And the doorway donor is always the wheel one position below the source (`src - 1`): its top bits are exactly the ones the slide pushed off its own top edge.



$\ll 74 = 64 + 10$, drawn with 8-bit wheels for visibility. Wheel $l[0]$'s content lands on $l[1]$'s doorstep, then slides 10 bits further — its top 10 bits (orange) cross the doorway into the next wheel's bottom.

Your own trace said it best: shifting by $2 \cdot 64 + 10$, wheel l_3 gets l_1 slid up and left by 10 — *plus* the top 10 bits of l_0 (its bits 63..54), fetched by shifting l_0 *right* by $64 - 10 = 54$. OR the two

pieces and the doorway is rebuilt. $\gg$ is the mirror: belt pulls right, beads fall off l_0 's right edge, zeros feed into l_3 's top.

The trap: when the leftover is 0 ($\ll 64$, $\ll 128$), the spill formula computes $\gg 64$ — and shifting a 64-bit value by 64 is **undefined behavior**: the hardware quietly masks the count, $\gg 64$ acts like $\gg 0$, and garbage leaks in through the doorway. Guard every spill term with `if (bit_shift && ...)`. It's the classic multi-limb bug: looks right, passes half the tests, corrupts exact-multiple-of-64 shifts.

6 Addressing: which wheel, which tick

`bit`, `byte_at`, `bit_length` all ask the same question: *given a bit index, which wheel and which position on it?*

```
wheel    = i / 64      which wheel holds bit i
position = i % 64      where on that wheel

bit(a, i) = (a.l[i/64] >> (i%64)) & 1    // slide to the end, mask
set bit i: q.l[i/64] |= 1 << (i%64)      // OR a one-bead mask
```

One deliberate quirk: `byte_at` counts from the **big end** — byte 0 is the most significant byte — because the EVM's `BYTE` opcode reads the number the way a human reads 4702, left to right. Everything else in the room counts from the small end. Keep the two conventions separate.

7 Multiplying: the area picture

Here's the moment it clicks. Don't think "multiply digits" — think **area**. $a \times b$ is the area of a rectangle with sides a and b . Cut each side into its four wheels and the big rectangle tiles into 16 small ones:

	a3	a2	a1	a0
b3	$a3 \cdot b3 \rightarrow 6$	$a2 \cdot b3 \rightarrow 5$	$a1 \cdot b3 \rightarrow 4$	$a0 \cdot b3 \rightarrow 3$
b2	$a3 \cdot b2 \rightarrow 5$	$a2 \cdot b2 \rightarrow 4$	$a1 \cdot b2 \rightarrow 3$	$a0 \cdot b2 \rightarrow 2$
b1	$a3 \cdot b1 \rightarrow 4$	$a2 \cdot b1 \rightarrow 3$	$a1 \cdot b1 \rightarrow 2$	$a0 \cdot b1 \rightarrow 1$
b0	$a3 \cdot b0 \rightarrow 3$	$a2 \cdot b0 \rightarrow 2$	$a1 \cdot b0 \rightarrow 1$	$a0 \cdot b0 \rightarrow 0$

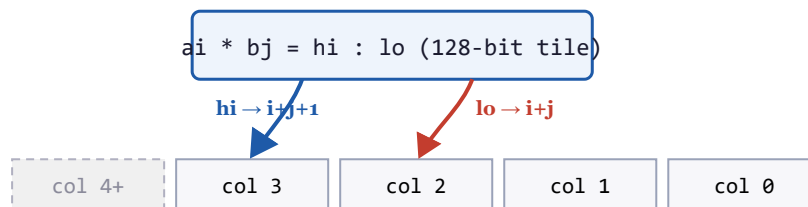
blue diagonal: all land on wheel 2 red tiles: column ≥ 4 — off the page

Each tile is labeled with the column its area lands on: tile (i, j) is a rectangle B^i wide and B^j tall, so its area carries the place value B^{i+j} . Place value isn't a rule — it's geometry. Red tiles land past wheel 3: skipped, and that's the mod.

Why do tiles on a diagonal share a column? Wheel a_i is worth B^i , wheel b_j is worth B^j , so tile (i, j) is a rectangle B^i wide and B^j tall: its area is $a_i b_j \cdot B^{i+j}$. The column index is the exponent — you never compute B^{i+j} , you just drop the tile's value into slot $i + j$.

One more physical fact: a wheel-times-wheel product doesn't fit on a wheel. $9 \times 9 = 81$ — a digit times a digit gives a *two-digit* answer. Ours is worse: 64-bit $\times$ 64-bit = up to 128 bits = **two wheels**. The hardware computes both halves at once; C++ hides the top half unless you ask.

`_umul128` is how you ask:



Each tile contributes its low half to column $i + j$ and its high half one column up. Tiles at column 4+ fall off the page.

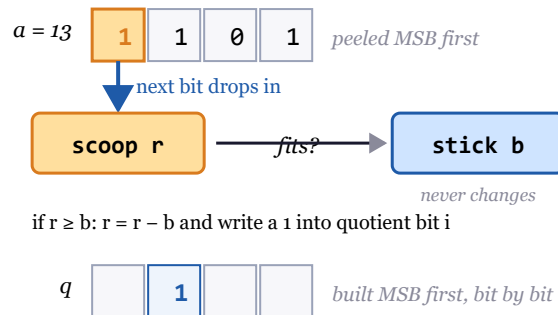
Walkthrough — $U(0, 1, 0, 0) \times U(0, 1, 0, 0)$, i.e. $2^{64} \times 2^{64}$: the only nonzero tile is $(i=1, j=1)$, so its *lo* lands on wheel 2, *hi* on wheel 3 (it's 0 anyway). Result: $U(0, 0, 1, 0) = 2^{128}$. ✓

Carries here are different from addition: the thing flowing upward isn't a flag, it's `hi` — a whole wheel's worth, up to $B - 2$. Why $B - 2$ max? The biggest tile is $(B - 1)^2 = (B - 2)B + 1$ — the "9 $\times$ 9=81, you can never carry a 10" argument.

8 Dividing: the measuring stick

Division answers a child's question: *how many times does the stick b fit along the rope a ?* In decimal long division you guess each digit — how many times does 37 go into 128? Binary removes the guesswork: **the quotient digit can only be 0 or 1**. Either the stick fits or it doesn't.

The machine: build the answer one bit at a time, most significant first. Keep a "remainder scoop" r . Each round, slide the scoop one place and drop the next bit of a into it — the "bring down the next digit" step from school. Then ask the only question: does the stick fit in the scoop? Yes $\rightarrow$ subtract it and write a 1; no $\rightarrow$ write a 0 and move on.



256 rounds: drop a bit into the scoop, ask "does the stick fit?" — subtract if yes, mark a 1.

Walkthrough — $13/3$ ($a = 1101_2$, $b = 0011_2$):

```

i=3:  scoop = 0<<1 | 1 = 1    stick fits? no          q = 0000
i=2:  scoop = 1<<1 | 1 = 3    fits? yes -> scoop = 0    q = 0100
i=1:  scoop = 0<<1 | 0 = 0    fits? no                 q = 0100
i=0:  scoop = 0<<1 | 1 = 1    fits? no                 q = 0100

q = 4, scoop left over = 1    check: 13 = 4*3 + 1    ✓

```

$r = (r \ll 1) \mid \text{bit}(a, i)$ is "slide the scoop, drop the next bit in." $\%$ is the same machine returning the scoop instead of the quotient.

Rulebook quirks: $x/0 = 0$, $x \bmod 0 = 0$. The clerk doesn't panic — division by zero is a defined answer, not an error.

9 Signed ops: same dial, different reading

No new machinery — `slt`, `sgt`, `sdiv`, `smod` are the clock-face section applied. Bit 255 = the sign flag. Signed compare: if signs differ, the negative one is smaller; same sign, compare as usual — remembering that among negatives, bigger magnitude means *more* negative. Signed division: unsigned-divide the magnitudes, then paint the sign back on — the quotient takes the XOR of the signs, the remainder follows the dividend's sign.

10 addmod / mulmod: the hidden wide desk

Trap ops. $(a + b) \bmod n$ looks like "add, then mod" — but $a + b$ can roll the odometer, and the rolled-over reading is a *different number* mod n :

```

addmod(MAX, MAX, 3):
  naive:  (MAX + MAX) wraps to  $2^{256} - 2$ ; mod 3 = 2    WRONG
  true:    $2^{257} - 2 \bmod 3 = 0$                         RIGHT

```

The clerk needs a hidden wider worksheet — a 257-bit scratch area for add, a full 512-bit one for mul — that never goes on the visible desk. `mulmod` is why the multiplication chapter matters so much: you need the **full** 512-bit rectangle before you can fold it mod n .

11 Conversions: the direction you read

Inside the room: little-endian wheels (wheel 0 is small). On paper, on the wire, in test vectors: big-endian — most significant byte first, like 4702. Every conversion is just walking in the right direction:

```
from_bytes(p):  p[0] is the BIG end -> top of wheel 3
                 p[n-1] is small end -> bottom of wheel 0
to_bytes32:      reverse walk, pad zeros on the LEFT
from_hex:        digits arrive big-end-first too
```

• Rules of the room

- The odometer has four wheels and no fifth: everything wraps mod 2^{256} .
- Carries and borrows between wheels are real; off the edge, forgiven.
- A wheel is a digit in base 2^{64} . Position IS place value.
- Wheel $\times$ wheel = two wheels. Quotient digits are only 0 or 1.
- The clerk never panics: $x/0 = 0$, and a clean stop is a valid answer.
- Signed and unsigned are two readings of one dial, not two numbers.